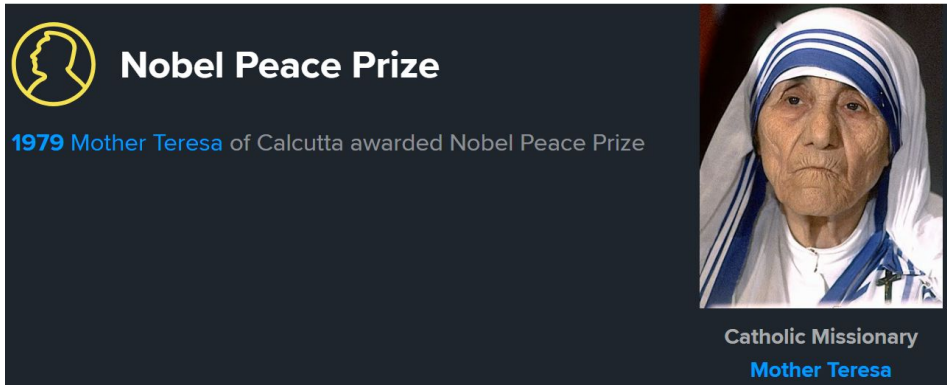


# COMS W4156 Advanced Software Engineering (ASE)

October 17, 2024

On this day ...



# Agenda

1. **First Assessment Soon**
2. if time permits, Logging
3. Reminders



# First Assessment Will Be Posted Friday October 25 12:01am

Deadline is Monday October 28 11:59pm, which means you will have four days minus two minutes to work on it

We expect the assessment to take about 4-6 hours to complete, but you can spend as much or as little time on it as you want, there is no "timer" when you start and your time spent does not need to be consecutive

The assessment can cover material discussed in class through today (or next Tuesday if we don't finish logging today), the assigned readings through today, the mini-project and the first iteration



# Is The Assessment Open Book?

You can read anything anywhere that you normally have access to

But do not post questions anywhere except to "staff" on edstem

Do not ask anyone or anything questions about the assessment except the teaching staff

Do not discuss the assessment with anyone or anything except the teaching staff

Do not discuss the assessment with ChatGPT or any other AI



# Graded vs Ungraded Questions



100 points total. There will be several graded questions clearly marked as such, e.g., "**part 1A (15 points)**", adding up to 100 points maximum score (minimum 0)

There will be two ungraded questions, "0A" and "0B"

0A is **required** in the sense that if you do not submit a complete answer, we will not grade your graded questions and you will receive 0 on the assessment

0B is **optional**, you can leave it blank

I will tell you the two ungraded questions now, and you can work on them before the exam is posted if you want. (The wording of the ungraded questions may change slightly, but the gist will be the same.)

## part 0A (required)

Allocate 100 total points among the members of your team, including yourself. Assign some number of points (possibly 0) to each member of your team based on what you believe was that student's relative participation in the team project thus far, and explain your scores.

Make sure the total is exactly 100. For allocating points, consider the level of effort and contributions for anything relevant to the team project.

Note equal participation is  
thing as equal lines of code produced.



same

Please do not predict the future, consider only the contributions to the team project as of the date of writing.

If your team uses pair programming, it is ok to allocate points to pairs instead of individuals.

## part 0A (required) 2

It is very important that you *explain* the scores you give to each student, including yourself: **If you only provide numbers, without meaningful explanations, then your answer will be considered incomplete and you will receive 0 overall on the assessment.**

Hint: "We all worked very hard and contributed equally" is not a meaningful explanation.



# Why Are We Asking You To Do This?

"Differential Grading" for team project (50% of total grade)

Default is for all members of the same team to get the same grade for the project

But every year there are a few students who contribute significantly less to their team project than their teammates

Such students may receive a lower grade than their teammates, possibly a *much* lower grade, based on input from their other team members (from ungraded questions on the first and second assessments) and their team Mentor

Differential grading is applied overall at end of semester, not per assignment



## Example (4 student team)

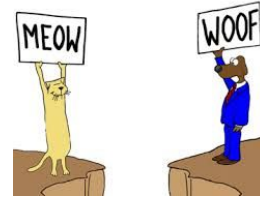


Overall, everyone had contributed a fair amount with a few minor differences here and there. Contributions from each member are listed below:

Me (32): Worked on setting up the ShrekDB database for persistent storage functionality. Also worked on the xxx, yyy and zzz classes. Added in unit test cases for xxx and yyy classes. Helped set up post\_this, post\_that and post\_the\_other\_thing endpoints. Also added API test cases to MailCarrier.

Greg (32): Set up ppp to qqg and rrr to qqg functionality using doppler and octachoron. Created functions to obtain qqg from ppp and rrr. Helped in setting up post\_that and post\_the\_other\_thing endpoints. Also helped with structuring the xxx class. Added API tests for ppp to qqg and rrr to qqg endpoints. Set up inside\_out and outside\_in endpoints. Helped set up style checkers.

## Continuing Example (4 student team)



Peter (21): Helped set up parrot for creating new endpoints for clients to access. Set up BloodTest and added initial test cases for yyy class. Helped set up style checkers. Helped create the yyy class functionality. Could have done a little more non-setup based coding.

Bobby (15): Setup main group repo. Helped find countless api for relocation. Worked on readme documentation for relocation api and citing sources. Felt like could have done a bit more work on different parts of the project and could have contributed to more of the code development but there were difficulties due to language barrier and he did try his best to contribute and collaborate to the extent that he could.

## Another Example (5 student team)



Me (25): Over the course of this project up to this point I was responsible for setting up the repository, in charge of leading the weekly team meetings along with coordinating everyone's schedules for meetings with our project mentor. I also implemented the waterbase service methods along with implementing the vast swath of api endpoints, I also made the Unit test class for the water base services. I also did the demo as well as set up our service on the cloud.

Alice (30): The best way to sum up Alice would be that she was the Tech Lead of the project, as the only group member among us who has professional experience as a software engineer, I recognized early that her input was vital to the success of the project. Whenever a team member had an issue with some technical aspect of the project, they would always consult with Alice first if she was available. Beyond that, Alice was responsible for setting up the continuous integration for our repository along with setting up the initial plugins and dependencies in the pompom file. She also set up the api endpoint skeleton code within our project's code base. Alice also initially tagged for the first iteration.

# Continuing Example (5 student team)



Marcia (15): Marcia was probably the least experienced in the realm of software development but she was very much eager to learn and took whatever tasks she was given to heart and gave it her all. She was primarily responsible for the creation of the aaa class in our project, which is an integral part of our service. Along with that she updated the teapot comments for the project as well as made most of the unit tests for the aaa class. She was also a part of our service's Quality Assurance team at the tail end of the first iteration. She also set up the Hopscotch board we used for the project!

Jan (15): Jan was very useful in writing up internal proposals and guidelines that were critical for ensuring our service was meeting the rubric specifications. She was also responsible for extending the functionality of the bbb class to handle things such as checking if two bbb objects were equal, as well as the ccc method that we used for one of our API endpoints. She has been plagued with not being able to make commits to the repo, so those respective features were committed under my name. Jan was also a member of the Quality Assurance team at the tail end of the first iteration.

# And Here's the Rest of This Example (5 student team)

Cindy (15): Cindy was responsible for the first stage planning of our service's demo with our mentor. She also made our entire repository style complaint by fixing the 730 warnings we had (a few of our XeroxStar instances didn't get the memo to switch from tabs to spaces so ~400 were tab warnings) nevertheless she managed to bring 730 warnings down to 0. She also was a member of the Quality Assurance team at the tail end of the first iteration.

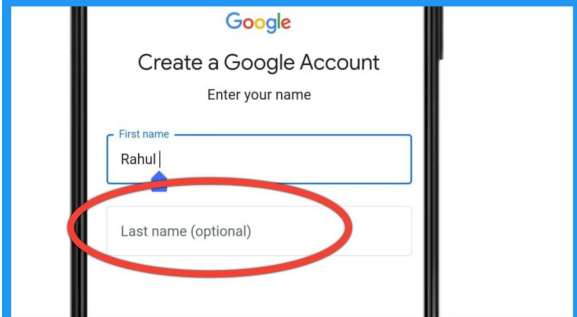
All members attended the regularly held meetings, Wednesdays 3:15am-4:45am where we discussed progress, deadlines, code design etc.



## part 0B (optional)

Please provide feedback on the course so far. Is there anything that you think should have been covered already but wasn't? Is there anything that was covered unnecessarily or should have been put off until later? Is there anything that should have been presented in a different way? Do you think starting with the individual mini-project was helpful, or should we have omitted the mini-project and spread the team project milestones over the entire semester? Is there anything else you would change about the course? Please explain, giving specific examples where warranted. Suggestions for the remainder of this course will be greatly appreciated.

Your team Mentor grades the  
but I read all the 0A and  
and adopt 0B suggestions  
(I ignore suggestions like "give everyone an A" 😁)



# Assessment Submission

You must submit a pdf file on courseworks. Submitting on edstem, by email, on paper (shudder!), or any other means except pdf on courseworks will receive 0

You can submit as many times as you like until the deadline. We recommend submitting a draft early, just in case

If you submit at exactly 11:59pm, courseworks will mark it "late" by 0 minutes (really!) but we will consider it on time

If you submit at exactly 12:00am, however, courseworks will mark it "late" by 1 minute and we will consider it late



# Ask More Questions NOW



# Agenda

1. First Assessment Soon
2. if time permits, Logging
3. Reminders



# Recording Beliefs for Debugging

Localizing a bug is a process of checking beliefs about the program

- Developer might believe that a function was called with particular parameters
- Developer might believe that a certain variable has a certain value at a certain place in the code
- Developer might believe that an API or library call returned a specific value
- Developer might believe that a particular execution path is taken through conditionals

You know what your beliefs are while writing the function, so add these beliefs to each function while writing it using logging

Comments help but logs are better, because they can be checked



# Code Beliefs

```
int foo (int a, int b) {  
    int c;  
    ...  
    code does something with a and b  
    ...  
    c = external_call(a,b);  
    ...  
    code does something with a, b and c  
    ...  
    return c;  
}
```



```
int foo (int a, int b) {  
    int c;  
        [ believe a>0 and b>0 ]  
    ...  
        [ believe a>0 and b>0 ]  
    c = external_call(a,b);  
        [ believe a>0 and b>0, believe c != 0 ]  
    ...  
        [ believe a>0 and b>0, believe c != 0 ]  
    return c;  
}
```

# Checking Beliefs During Debugging



## Six Stages of Debugging

1. That can't happen.
2. That doesn't happen on my machine.
3. That shouldn't happen.
4. Why does that happen?
5. Oh, I see.
6. How did that ever work

If you execute the function in an [interactive debugger](#) *soon after writing it when you still remember all your beliefs*, you can manually check that your beliefs are correct for a small number of test cases. Set breakpoint at beginning of function and examine the values of the parameters, then single-step through code while examining values that could have changed

More commonly interactive debugging happens later, after tests fail, and the developer (or your future self) may not know what the beliefs were unless something present in the code tells them

They cannot know whether those beliefs were correct during previous executions unless saved execution logs tell them

# Code Beliefs

```
int foo (int a, int b) {  
    int c;  
    ...  
    code does something with a and b  
    ...  
    c = external_call(a,b);  
    ...  
    code does something with a, b and c  
    ...  
    return c;  
}
```

```
int foo (int a, int b) {  
    int c;  
        [ breakpoint to check a>0 and b>0 ]  
    ...  
        [ breakpoint to check a>0 and b>0 ]  
    c = external_call(a,b);  
        [ breakpoint to check a>0, b>0, c != 0 ]  
    ...  
        [ breakpoint to check a>0, b>0, c != 0 ]  
    return c;  
}
```



# Logging Records Belief Checks Automatically

Use logging package instead of print statements!

Log entries are just messages that describe events or state during execution

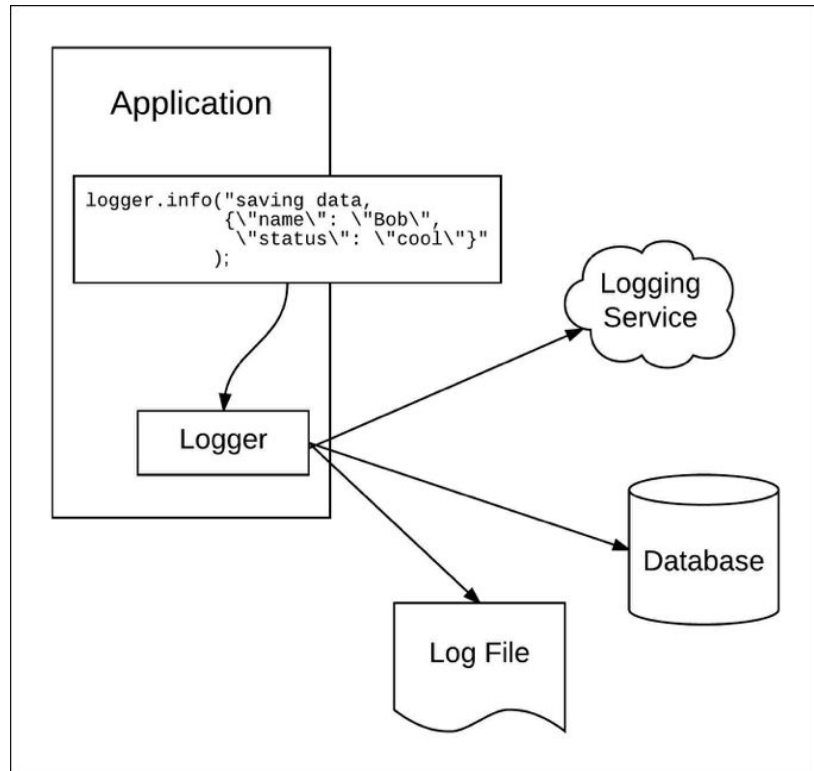
Severity levels might be configured so all levels are logged during development with only the most severe levels logged in production - the other logging statements are not necessarily removed, they just don't do anything

Turning off some log statements during production reduces log size and logging overhead

```
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
WARN  LoggingUtilAgent - I am concerned...
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
INFO  LoggingUtilAgent - I am happy!
DEBUG LoggingUtilAgent - I am up, I am down, I am all araound!
INFO  LoggingUtilAgent - I am happy!
INFO  LoggingUtilAgent - I am happy!
WARN  LoggingUtilAgent - I am concerned...
INFO  LoggingUtilAgent - I am happy!
INFO  LoggingUtilAgent - I am happy!
DEBUG LoggingUtilAgent - I am up, I am down, I am all araound!
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
WARN  LoggingUtilAgent - I am concerned...
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
INFO  LoggingUtilAgent - I am happy!
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
ERROR LoggingUtilAgent - I am in trouble, something went wrong.
DEBUG LoggingUtilAgent - I am up, I am down, I am all araound!
INFO  LoggingUtilAgent - I am happy!
DEBUG LoggingUtilAgent - I am up, I am down, I am all araound!
```

# Logging Beliefs

```
int foo (int a, int b) {  
    int c;  
    log("beginning of foo",a,b);  
    ...  
    log("middle of foo",a,b);  
    c = external_call(a,b);  
    log("after foo calls external_call",a,b,c);  
    ...  
    log("end of foo", a,b,c);  
    return c;  
}
```

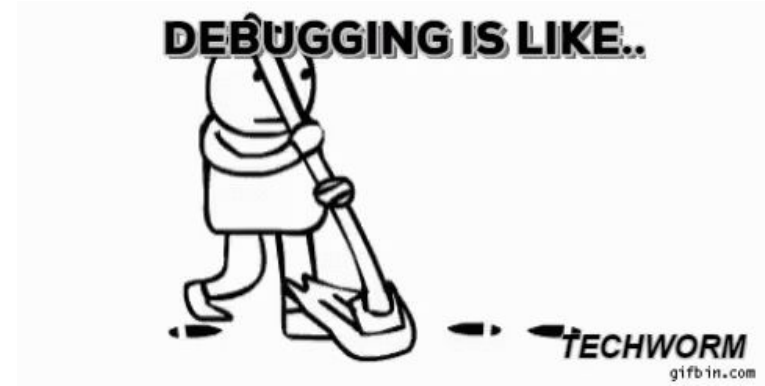


# Why Not Use Print Statements?

What is wrong with print statement debugging?  
It's what everyone learned first, it "worked",  
why give it up?

Introducing print statements *changes the code* - modern languages are designed to prevent semantic changes due to built-in (or standard library) debugging statements, but cannot help if you introduce new code that cannot be distinguished from program code

This problem is most significant with C/C++ and other languages with unmanaged memory, but can occur in nearly any language



# What Are Severity Levels?

By setting the [log level](#) appropriately, developers can filter out irrelevant information and focus on what's important

- **Debug:** Show diagnostic information about the system.
- **Info:** Information about major events in a system's regular operation, such as a successful system startup or shutdown.
- **Warning:** Runtime issues that do not result in an error, but may indicate other problems that could eventually result in an error, such as system instability or connectivity issues.
- **Error:** Runtime issues that violate the requirements of a system, such as unhandled input or outputs to the process.
- **Critical:** Events that violate the system's requirements such that they will generally result in a system crash.

Setting the log level too low can result in missing important information; putting it too high can generate excessive noise, making it harder to find essential logs and slowing down the system



# Generate Diagnostic Information

When debug logging, you will want to record *detailed* diagnostic information to make it easier to understand what your program is doing:

- Timestamps of events
- (Partial) ordering of events across different threads/processes
- Resource usage - what, when, how much
- Entity IDs and Session IDs for communicating entities
- Data received from/sent to those entities
- Values of key variables



# Benefits of Debug Logging



**Assistance in identifying root cause:** When an error or unexpected behavior occurs, developers can examine the log files to understand what happened and [identify the root cause](#) of the issue. Debug logging can provide a wealth of information about the system's state at the time of the error, including the values of variables and other in-memory data, the state of the environment, and the sequence of events that led to the error.

**Insight into how to fix an error:** Just as debug logging information helps isolate the root cause of an error, that same information provides insights into how developers can implement a fix, e.g., what code was running when the bug manifested (the log diverged from expected behavior)?

**Insight into how to reproduce an error:** When edge cases are hit during runtime, the information gleaned from debug logging can make [bug reproduction](#) easier

**Early identification of potential performance bugs:** By logging data during runtime, such as time of request and time of corresponding response, developers can better understand a program's performance and identify potential bottlenecks or areas for optimization

# Protect Sensitive Log Data



It is essential to protect sensitive information when logging in production environments or when production data is used in testing

For example, debug logs accessible to ordinary developers should not record personal identifying information ([PII](#))

When PII data must be logged, e.g., audit logs for security purposes, the data should be secured through log file permissions and securely shipped to an external log repository

# Some Logging Packages

[loggly ultimate guide](#) covers many languages and platforms

[logging for C++](#)

[logging for Java](#)



# Agenda

1. First Assessment Soon
2. if time permits, Logging for Debugging
3. Reminders



# Deadlines

[Team First Iteration](#) due **tomorrow** October 18

[Team First Iteration Demo](#) due the following Friday October 25

First Assessment

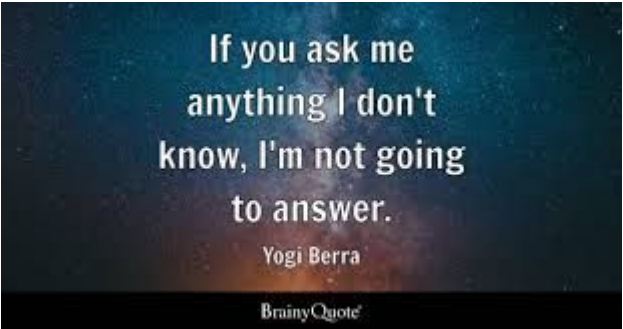
Friday October 25th 12:01am through Monday October 28th 11:59pm



# Next Week

TBD





If you ask me  
anything I don't  
know, I'm not going  
to answer.

Yogi Berra

BraintyQuote

# Ask Me Anything